Lesson notes · for you

How work grows: big O

Programming · Level 4 · Lesson 1

Time
20 min

Materials
A phone with this lesson downloaded; A notebook for the step counts

Learning goals

- Count the steps an algorithm takes as the input grows.
- Classify code as O(1), O(log n), O(n), O(n log n) or $O(n^2)$.
- Replace an $O(n^2)$ method with a faster one.

Activity: Activity: Every pair, or one pass?

1. Warm-up (3 min): Write down a guess: how long would comparing every pair of a million records take on a phone?
2. Count (7 min): With paper, count the comparisons every-pair needs for 4, 8 and 16 items. What happens each time n doubles?
3. Improve (7 min): Write the set version and count its checks for the same sizes.
4. Explain (3 min): Write both as $O(n^2)$ and O(n), then check your warm-up guess with ten million steps a second.

Counting on paper is enough: the pattern appears by 16 items.

Discussion

- Why do computer scientists count steps instead of seconds?
- Where have you seen a hidden loop?
- When is $O(n^2)$ good enough?

Check yourself

- Write a loop that is O(n), and one that is $O(n^2)$, for the same list.

Try it this week
Find a shared birthday among ten friends two ways: compare every pair, then go round once writing each birthday on a list you check. Count the comparisons each way.

Worksheet

How work grows: big O

Programming · Level 4 · Lesson 1

Name _____ Class _____ Date _____

1 **A duplicate check compares every pair, and it is too slow for a million records. What helps most?**

◯ Run it on a faster phone ◯ Remember the values seen in a set ◯ Compare every pair twice

2 **Write first_repeat(values): it returns the first value that appears a second time, or None. It must stay fast for 20,000 values, so use a set, not a loop inside a loop.**

```
def first_repeat(values):
    # return the first value seen twice, or None
    # (a set keeps it fast: one pass)
    return None
```

Write your program:

3 **What is the big O of this function, for a list of n values?**

```
def has_pair(a, target):
    for x in a:
        for y in a:
            if x + y == target:
                return True
    return False
```

◯ O(1) ◯ O(log n) ◯ O(n) ◯ $O(n^2)$

4 **Fill the gaps: the program counts how many times 1,000 can be halved before it reaches 1.**

```
n = 1000
steps = 0
while n _____ 1:
    n = n _____ 2
    steps = steps + 1
print(steps)
```

Word bank: - < // >

5 **Match each piece of code to its big O.**

Read a[0]	• • • •	$O(n^2)$
Binary search on a sorted list	• • • •	O(n log n)
One loop over the list	• • • •	O(n)
Merge sort	• • • •	O(log n)
A loop inside a loop over the list	• • • •	O(1)

✎ **Find a program you wrote with a loop inside a loop. What is its big O? Could a set or a dictionary remove the inner loop?**